The Essential of Chapter 5

Query optimization cut down to what the two mock exams ask: selectivity arithmetic, estimated against actual, the cost of a scan and of an index, why the estimate is the hard part, the levers when the plan is wrong

Advanced Data Management

2026-10-29

How to read this document

Chapter 5 with everything removed that neither mock exam nor the chapter 5 quiz touches. Six sections, each ending in a *Mock drill* box naming the mock and quiz questions that live there and the shape of the rubric answer. Numbers to carry, all from `adm_lab`: `orders` 500,000 rows in 3,185 pages (157 rows per page), `customers` 10,000 rows in 73 pages, `products` 200 rows in 2 pages; `amount` uniform on 5–105 with ten equi-depth buckets at $5, 15, \ldots, 105$; `cust_id` 10,000 distinct values, `product_id` 200; costs: sequential page 1, random page 4, row 0.01, operator 0.0025, index entry 0.005. **Left out on purpose:** the pipeline figure and the other logical rewrites (§1–2), rule-based versus cost-based (§7.1), interesting orders, the DP lattice and genetic search (§6), the Lab 5 companion (§8) and problems P5–P8 (§9).

1 What the Optimizer Decides

Concept: Logical plan, physical plan, planner

The parser turns SQL into a tree; the rewriter hands the optimizer a **logical plan**, *what* to compute: a tree of σ (selection), π (projection), $\bowtie$ (join), γ (grouping). The **optimizer**, PostgreSQL's **planner**, turns it into a **physical plan**, *how*: which scan per table, which algorithm per join, in which order. The executor runs the plan it is given; the buffer manager serves page requests and never sees the SQL.

The optimizer chooses *without running anything*: it has only the **catalog**, the statistics `ANALYZE` collected. It **estimates** how many rows each operator will see (§2), **costs** each candidate from those sizes (§4) and **chooses** the cheapest join order (§6). Before that, the rewriter applies **selection pushdown**, $\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ when p mentions only R: a filter on

`customers.country` moves below the join, which then sees 2,000 customers instead of 10,000. A *logical* rewrite, applied always because it needs no statistics; in a plan it is the `Filter:` on the `Seq Scan` *under* the join. Hash versus merge, index versus scan are *physical* choices, made by costing.

Mock drill

Mock v2 A1: the planner decides Seq Scan versus Index Scan (b), by costing both from the statistics; parser builds the tree, executor runs the plan, buffer manager serves pages. **Quiz A7**: pushing `WHERE o.amount > 100` below the join is the logical rewrite (b); the other three are physical. One sentence each: name the component or the identity, say what the others do.

2 Selectivity Arithmetic

`ANALYZE` samples each table and stores what every estimate is built from: in `pg_class` the counts `reltuples` and `relpages` ($|R|$ and B_R); in `pg_stats`, per column, `n_distinct`, the **most common values** with their frequencies (the MCV list, which is how skew is captured), the **histogram** bounds for ranges, and the `correlation` between physical row order and the column (-1 to 1, §4). `country = 'MA'` with MCV frequency 0.2 estimates $0.2 \times 10{,}000 = 2{,}000$ customers.

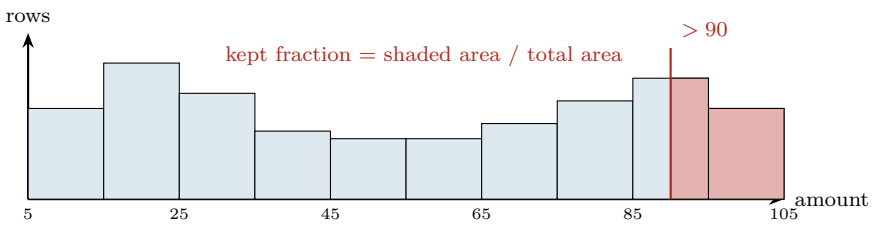
2.1 One predicate

Concept: Selectivity

The **selectivity** of a predicate p on R is the fraction of rows that satisfy it, $\text{sel}(p) = |\sigma_p(R)|/|R| \in [0, 1]$: the probability that a random row passes. Estimated rows $= |R| \cdot \text{sel}(p)$; estimated pages = rows / rows per page.

Equality. If the constant is in the MCV list, its stored frequency is the selectivity. Otherwise the rows are assumed spread uniformly over the distinct values and the selectivity is 1/n_distinct. `cust_id = 42`: 500,000/10,000 = 50 rows. `product_id = 7`: $1/200 = 0.005$, 2,500 rows.

Range. Needs the distribution: the **equi-depth histogram**, k bucket boundaries chosen so that every bucket holds the same number of rows, $1/k$ of them (100 by default, 10 for the lab's `amount`).



Mechanism: Range selectivity from an equi-depth histogram

For `col > v`: every bucket entirely above v contributes $1/k$; the bucket containing v contributes $1/k$ times the fraction of its width above v (uniform inside a bucket). For `BETWEEN`, count the buckets between the two constants the same way. On `amount` (width 10, 10% each): `> 95` is one whole bucket, $s = 0.10$, 50,000 rows, $50{,}000/157 \approx 318$ pages if packed; `BETWEEN 20 AND 40` is half, whole, half, $0.05 + 0.10 + 0.05 = 0.20$, 100,000 rows (the whole bucket alone is the trap answer 10%).

2.2 AND, OR, and the independence assumption

$$\text{sel}(p \wedge q) = \text{sel}(p)\,\text{sel}(q), \qquad \text{sel}(p \vee q) = \text{sel}(p) + \text{sel}(q) - \text{sel}(p)\,\text{sel}(q).$$

The AND rule is the **independence assumption**: the columns are treated as unrelated, as probability multiplies for independent events; OR is inclusion–exclusion under the same assumption. Never *add* selectivities for an AND. When p *implies* q (every Casablanca customer is in MA) the truth is $\text{sel}(p)$ and the product is smaller: correlation in an AND *under*-estimates, always, by the factor $\text{sel}(q)$ multiplied in for nothing.

Worked example: Mock v2 A6: `amount > 95 AND product_id = 7`

Range: $(105-95)/(105-5) = 0.10$. Equality, no MCV list: $1/200 = 0.005$. AND under independence: $0.10 \times 0.005 = 0.0005$; $0.0005 \times 500{,}000 = \mathbf{250}$ rows. Traps: 50,000 is the range alone; 52,500 is 0.10+0.005, the OR/AND confusion (the OR itself is 0.1045, 52,250 rows).

2.3 Join size

For $R \bowtie_{R.a=S.b} S$ with d_R distinct values of a and d_S of b:

$$|R \bowtie S| \approx \frac{|R| \cdot |S|}{\max(d_R, d_S)}.$$

The side with more distinct values is assumed to contain every value of the other, so each row of the other side matches $|R|/d_R$ rows on average. Not min, not $d_R + d_S$. **Key–foreign key:** if $R.a$ is a primary key, $d_R = |R|$ and the formula collapses to $|S|$: every order matches one customer, so customers $\bowtie$ orders has 500,000 rows; after a filter keeping a fifth of the customers, $2{,}000 \times 500{,}000/10{,}000 = 100{,}000$. **Keep the denominator honest:** filtering `products` to 10 ids leaves `orders` with 200 distinct `product_id`, so the join is $10 \times 500{,}000/\max(10, 200) = 25{,}000$. No join predicate: a cross product, $|R| \cdot |S|$.

Mock drill

Mock v2 A6: the 250-row example, three factors and the word "independence". **Mock 1, 3.5(a)**: `country = 'MA'` (0.2) `AND city = 'Casablanca'` (0.1) on 2,000,000 users: independence $0.2 \times 0.1 \times$

2,000,000 = 40,000; truth, since city implies country, $0.1 \times 2{,}000{,}000 = 200{,}000$, "5× too low"; fix in §3. **3.5(b)**: `'view'` 0.9 and `'purchase'` 0.005 from the MCV list. **Quiz A1** (50), **A2** (20%), **A5** (too low), **B1(a)**, **B2(a)–(b)** (20 against 3,000; join 1,000 against 150,000), **B3(a)–(b)** (25,000 and 200,000). Shape: formula, substitution, one sentence quoting the number.

3 Reading a Plan: Estimated Against Actual

```
Seq Scan on orders  (cost=0.00..9435 rows=50000) (actual time=0.01..61.2 rows=49870
    loops=1)
  Filter: (amount > 95)
  Rows Removed by Filter: 450130
```

`cost=startup..total`: the planner's estimate in cost units (§4). `rows`: *estimated* output rows. `actual rows`: measured, printed only by `EXPLAIN ANALYZE`, which really runs the query. `loops`: how many times the node ran; a nested loop's inner runs once per outer row and its `actual rows` is *per loop*.

Mechanism: q-error, and where an estimate comes from

The quality of an estimate is its **q-error**, $\max(\text{est}/\text{act},\ \text{act}/\text{est})$, always ≥ 1 and symmetric. Under 2 is good; 10 is the red flag. An *under*-estimate is the dangerous direction: a believed-small input invites a nested loop or a hash build that will not fit. **Mock v2 C2**: `rows=43388` against `actual rows=42299` is $\max(43{,}388/42{,}299,\ 42{,}299/43{,}388) = 1.026$, a 2.6% overestimate. Its source: the fraction of the `order_date` histogram at or above `2025-12-01`, about 8.7%, times `reltuples` = 500,000; accurate because the statistics are fresh and a single-column range on a histogram is the easy case; the key–foreign-key join above inherits it.

3.1 Estimates propagate upward

Every operator's estimate is computed from its children's; a plan is priced bottom-up, so the leaves feed everything above them.



If the leaf is wrong by ten (2,000 believed, 20,000 real), the join is wrong by ten, the hash table may spill, and worse: the planner may have chosen a nested loop *because* it believed the outer had 2,000 rows. Errors do not cancel; through joins they multiply.

Key insight. The lowest wrong node is the disease; a nested loop with a huge `loops`, a `Hash` with `Batches` > 1, an external sort are symptoms. Fix the leaf and the plan above changes on its own.

3.2 The three estimation failures

Worked example: Stale (`ANALYZE`), correlated (`CREATE STATISTICS`), skewed (MCV list)

1. Stale. Autovacuum re-analyses only after about 10% of the rows changed; a bulk load outruns it. Add 300,000 orders without analysing:

```
before ANALYZE:  rows=4873   actual rows=7800    <- scaled to the old 500,000
after  ANALYZE:  rows=7790   actual rows=7800    <- row count and histogram
     rebuilt
```

The table grew 60%, so every estimate was 60% low. Signature: the base count itself is wrong.

2. Correlated columns. Per-column statistics are right; the *product* is wrong.

```
CREATE STATISTICS s_geo (dependencies) ON country, city FROM
    customers;
ANALYZE customers;    -- the object is empty until analysed
```

```
before:  rows=400    actual rows=2000    <- 0.2 x 0.2 x 10,000 = 400
after:   rows=1950   actual rows=2000    <- dependency city -> country now known
```

`ANALYZE` alone changes nothing here; the extended statistic measures how strongly `city` determines `country`, and the planner stops multiplying.

3. Skew. If one value dominates, 1/n_distinct is too low for it and too high for the rest. The MCV list fixes that; trouble starts when the heavy value is not in it (`ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000`) or when the constant is a parameter (`col = $1`): `'purchase'` at 0.5% wants an index, `'view'` at 90% a scan.

Worked example: Mock 1, 3.1: the 13-minute plan

`Nested Loop rows=50 actual rows=1,240,000`; under it `Seq Scan on users rows=40 actual rows=200,000, Filter: city = 'Casablanca'`; the inner side a `Seq Scan on events` (1,000,000 pages) once per outer row. **(a)** Most wrong node: the leaf scan on `users`, 200,000/40 = 5,000×; the join's 24,800× is that error multiplied upward. **(b)** Stale statistics (`ANALYZE` last ran at 5M events; there are 50M) and correlation plus skew (`city` determines `country`; `event_type` is 90/9.5/0.5). **(c)** `ANALYZE users; ANALYZE events;` then `CREATE STATISTICS s_geo (dependencies) ON country, city FROM users; ANALYZE users;` then `ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 1000;` and re-explain. With 200,000 believed outer rows the planner switches to a hash join: build on the filtered users (about 3,000 pages, fits `work_mem` of 8,000), one scan of `events`: 30,000 + 1,000,000 pages read once, 16 s at 500 MB/s instead of 13 minutes.

Mock drill

Mock v2 C2: q-error 1.026, the source, why accurate. **Mock 1, 3.1**: lowest wrong node and its factor, two causes, commands in order, the join that results. **Quiz A4** (q-error 1,000, an under-estimate), **A8** (stale, `ANALYZE orders`), **B2(c)–(d)** (`ANALYZE` does not help; the two statements that do), **C2–C3** (`rows=4` = 50×0.0868; q-error 11; a per-customer pattern no statistic describes). Shape: number, the failure by name, the command, what the plan becomes.

4 The Cost Model and the Access Path

A **cost** is one number per plan, a weighted sum of estimated I/O and CPU work in arbitrary units, so that plans for the *same* query can be compared; it is not time. PostgreSQL's weights:

Parameter	Default	Charged for
`seq_page_cost`	1.0	one page read in a sequential scan (the unit)
`random_page_cost`	4.0	one page fetched at random (index lookups into the heap)
`cpu_tuple_cost`	0.01	processing one row
`cpu_operator_cost`	0.0025	one operator evaluation (a filter comparison)
`cpu_index_tuple_cost`	0.005	processing one index entry

Sequential scan of B pages, N rows: $B \cdot 1 + N \cdot 0.01$, plus $N \cdot 0.0025$ when there is a `Filter` (every row is compared). Startup cost 0. **Plain index scan**, h levels, m matches, *unclustered* table (each match on its own heap page): $h \cdot 4 + m \cdot 4 + m \cdot (0.005 + 0.01)$: the descent, one random page per match, one index entry and one row of CPU per match.

Worked example: `orders`, 3,185 pages, 500,000 rows, 3-level index on `amount`

`EXPLAIN SELECT * FROM orders`: $3{,}185 + 500{,}000 \times 0.01 = 8{,}185$. With `WHERE amount > 95`: $3{,}185 + 5{,}000 + 1{,}250 = 9{,}435$. Index scan for the same 50,000 matches: $3 \times 4 + 50{,}000 \times 4 + 50{,}000 \times 0.015 = 12 + 200{,}000 + 750 = 200{,}762$, twenty-one times the scan: the rows sit wherever they were inserted, so every match is a random page.

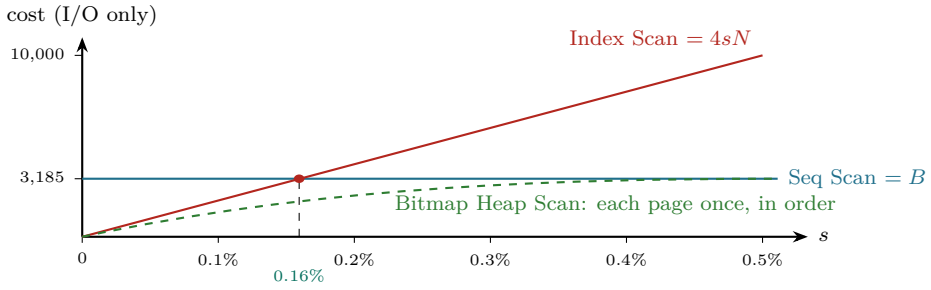
4.1 When the index wins

Ignore the CPU terms: the scan costs B whatever the selectivity s; the index costs about $4sN$ random pages. The index wins when

$$4sN < B \quad \Longleftrightarrow \quad s < \frac{B}{4N} = \frac{1}{4 \cdot \text{rows per page}}.$$

On `orders`, $1/(4 \times 157) \approx 0.16\%$, about 800 rows, roughly `amount > 104.84`; with 100 rows per page, 0.25%. A predicate keeping 10% or 40% is far above

it: *an index on a predicate that keeps a large fraction of the table is never used, and the planner is right.*



Correlation and bitmap scans. The $4sN$ line assumes every match is on a different page, which is what `correlation` ≈ 0 means. PostgreSQL softens the cliff with the **bitmap heap scan**: collect the matching row ids from the index, sort them by page, read each page once in page order. It pays only for the *distinct pages* holding a match, still nearly all here: with 8.5% kept and 157 rows per page, a page has no match with probability $(1-0.085)^{157} \approx e^{-13.9} \approx 10^{-6}$, so all 3,185 pages are read anyway. Hence the flip happens between 1% and 10% in practice, higher when the correlation is near ± 1: a table physically ordered by the column puts 8.5% of the rows in 8.5% of the pages, about 271, and an index (or BRIN) wins by ten. `random_page_cost` $= 1$ (SSD) moves the threshold to $1/157 \approx 0.64\%$.

Mock drill

Mock 1, 1.2: 500,000 rows, 5,000 pages, 40% kept: $200{,}000 \times 4 = 800{,}000$ against 5,000, sequential scan (b); threshold $1/(4 \times 100) = 0.25\%$. **Mock v2 C4**: index on `order_date`, correlation -0.0015, 8.5% kept: 13 matches per page, 10^{-6} chance of an empty page, 12,740 against 3,185, no; it would help with correlation near 1 (271 pages, BRIN). **Quiz A3** (8,185), **B1** (9,435 against 200,762, the 0.16% rule, correlation 0.002), **C4–C5** (`shared hit=55` $= 3 + 52$, one heap page per order), **D1**. Shape: both costs with their terms, the inequality, the decision in one sentence.

5 Why Estimation Is the Hardest Part

The optimizer rewrites, estimates, costs and searches. Three of the four are solved: rewriting is mechanical and always answer-preserving; costing is arithmetic, exact *given its inputs*; join enumeration is a textbook dynamic programme (§6). All three take row counts as input, and the row count of anything but a base table must be *predicted* from a distinct count, a hundred histogram bounds, a short MCV list and the assumption that columns are independent. That prediction, on a skewed, correlated, changing dataset, is the only genuinely uncertain step, and the usual root cause of a bad plan is a bad estimate, not a bad search. One wrong number is expensive because every later decision is a *function* of the estimated sizes: join algorithm, join order

and access path all flip at some row count, and a q-error of 10 at a leaf crosses several thresholds at once. The lecture's dashboard: a nested loop chosen on a believed 2,000-row outer, really 20,000, ran 45 minutes; one `ANALYZE` later, 20 seconds.

In the real world. Quiz C3 shows the limit: for one customer, 45 of 52 orders fall in December, and no catalog statistic describes one customer's calendar. Some estimates are beyond repair, which is why §6 keeps a hint extension as the last lever.

Mock drill

Mock 1, 2.5 is this section in three paragraphs: the claim (the other three steps are mechanical, estimation is the uncertain one); propagation with a number (40 believed against 200,000 real is 5,000× at the leaf, the same factor at every node above, a different algorithm at each); two causes with fixes (stale, `ANALYZE`; correlation, `CREATE STATISTICS`). **Quiz C6**: a q-error of 11 is harmless when both alternatives are cheap, a disaster when it sizes a nested-loop outer or a hash build. **Quiz D2** opens the same way.

6 Join Ordering and the Levers

Commutativity and associativity make the join order free, and in a big join it matters more than anything else: joining two large unfiltered tables first builds an intermediate another order never materialises. A **left-deep** plan adds one base table per join; even so there are $n!$ orders, $5! = 120$.

Mechanism: System-R dynamic programming

The best plan for a set of tables contains the best plan for each of its subsets, so solve each subset once. **Level 1**: the cheapest access path per table. **Level** k: for each subset of k tables, the cheapest of (best plan of $k-1$ of them) $\bowtie$ (the remaining table), over every join algorithm. **Level** n: the answer. Subsets are memoised, not sequences: $2^n - 1$ of them (31 for five tables) instead of $n!$ orders. Pairs with no join predicate are deferred while a connected pair exists.

Quiz B3 runs it on filtered customers (4,000 rows), orders (500,000) and filtered products (10 of 200): products $\bowtie$ orders first sends 25,000 rows into the second join against 200,000 for customers $\bowtie$ orders; the cross product is never costed; the result is 10,000 either way. PostgreSQL reorders the tables of one `FROM` list up to `join_collapse_limit` (default 8); beyond it, or at 1, it keeps the order you wrote, which is how the lecture's six-table report joined two big tables before its selective filter.

6.1 No hints: the levers in order

PostgreSQL has no optimizer hints by design: every lever improves what the planner *knows*, so every query benefits, not only the one that hurt today. The order the exam wants:

1. **Statistics**: `ANALYZE`; then `CREATE STATISTICS (dependencies)` on cor-

related columns; then a larger statistics target so the skewed value enters the MCV list.

2. **Cost parameters**: on SSD lower `random_page_cost` to 1.1, and tell the planner the real cache size (`effective_cache_size`); try per session, then set per server.
3. **Memory**: `work_mem` so the right hash join or sort does not spill (`SET LOCAL` for one statement, not a global jump).
4. **Experiments**: `SET enable_nestloop = off` in *one session* to see the second choice, then `RESET`: the `enable_*` switches are a 10^{10} cost penalty, not a prohibition, and server-wide they regress every other query.
5. **Last**: the `pg_hint_plan` extension, for data no statistic can describe. A hint freezes one plan for one distribution; next year's tenfold table makes it wrong.

Mock drill

Quiz A6 (120 and 31), **B3(c)–(d)** (products first, cross product deferred, `join_collapse_limit`), **D2** (the five levers in order, why hints rot). **Mock v2 B3(d)** and **D2**: `enable_*` is a penalty for experiments, not a production switch. Ticket shape: claim, mechanism, number, consequence.

Warning / Gotcha: The four mistakes that cost points

- Adding selectivities for an AND (52,500 instead of 250), or calling the independence error "too high". AND multiplies; correlated predicates make the product an *under*-estimate.
- Dividing the join size by min or $d_R + d_S$, or shrinking the denominator after filtering the other side. It is $\max(d_R, d_S)$; key–foreign-key gives the foreign-key side.
- Forgetting a cost term: 3,185 alone, 8,185 with a `Filter`, or $3{,}185 \times 4$ for a sequential scan.
- Treating the symptom: an index or `enable_hashjoin = off` when `rows` is 1,000 times off. Fix the lowest wrong node: `ANALYZE` first, `CREATE STATISTICS` second.

Summary: The essential in five lines

1. The planner picks the physical plan without running anything, from catalog statistics; selection pushdown is the logical rewrite applied first.
2. Equality: MCV frequency or 1/n_distinct; range: equi-depth buckets, $1/k$ each, partial bucket by width; AND multiplies (independence), OR adds and subtracts the product; join $|R||S|/\max(d_R, d_S)$, $|S|$ for key–foreign-key.
3. q-error $= \max(\text{est}/\text{act}, \text{act}/\text{est})$, under 2 good, 10 a red flag; errors multiply upward, so fix the lowest wrong node: stale (`ANALYZE`), correlated (`CREATE STATISTICS`), skewed (bigger MCV list).
4. Scan $= B + 0.01N$ $(+\,0.0025N)$, index $= 4h + 4m + 0.015m$; the index wins below $1/(4 \cdot \text{rows per page}) \approx 0.16\%$; bitmap scans and correlation move the flip to 1–10%.
5. Estimation is the hard part because every later decision is a function of row counts; System-R DP searches 2^n subsets, not $n!$ orders; no hints: statistics, costs, `work_mem`, `enable_*`, `pg_hint_plan` last.